

08-计算机网络

计算机网络面试题

1. 网络模型

问：TCP/IP 四层模型和 OSI 七层模型？

答：

OSI 七层	TCP/IP 四层	协议
应用层	应用层	HTTP、HTTPS、DNS、FTP
表示层	↑	
会话层	↑	
传输层	传输层	TCP、UDP
网络层	网络层	IP、ICMP、ARP
数据链路层	网络接口层	以太网
物理层	↑	

2. HTTP/HTTPS

问：HTTP 1.0、1.1、2.0、3.0 的区别？

答：

版本	特点
HTTP/1.0	短连接，每次请求新建 TCP
HTTP/1.1	长连接（Keep-Alive）、管道化、分块传输
HTTP/2.0	多路复用、头部压缩（HPACK）、服务器推送、二进制分帧
HTTP/3.0	基于 QUIC（UDP），解决 TCP 队头阻塞

问：HTTPS 握手过程？

答：

1. Client Hello: 客户端支持的 TLS 版本、加密套件、随机数
2. Server Hello: 选定加密套件、服务器证书、随机数
3. 客户端验证证书 (CA 链), 生成 pre-master secret, 用服务器公钥加密发送
4. 双方根据三个随机数生成会话密钥 (对称加密)
5. 后续通信用对称加密

问: 常见 HTTP 状态码?

答:

状态码	含义
200	成功
301/302	永久/临时重定向
304	未修改 (协商缓存命中)
400	请求参数错误
401	未认证
403	无权限
404	资源不存在
500	服务器内部错误
502	网关错误
503	服务不可用

502 vs 504:

状态码	含义
502 Bad Gateway	网关/代理从上游收到 无效响应 (上游挂了或返回格式错误)
504 Gateway Timeout	网关等待上游 超时 (上游慢或未响应)

问: GET 和 POST 的区别?

答:

对比	GET	POST
参数位置	URL Query	Body
幂等性	幂等	非幂等
缓存	可缓存	一般不缓存
长度限制	URL 长度限制	无限制 (理论)
安全性	参数暴露在 URL	相对安全

3. TCP（最高频）

问：TCP 三次握手过程？

答：

```
1. 客户端 → SYN (seq=x)           状态: SYN_SENT
2. 服务端 → SYN+ACK (seq=y, ack=x+1)  状态: SYN_RCVD
3. 客户端 → ACK (ack=y+1)           双方: ESTABLISHED
```

第三次握手可携带数据，前两次不可。

问：TCP 报文头部有哪些关键字段？

答：

字段	作用
源/目的端口	16 bit, 标识应用
序列号 seq	保证有序、去重
确认号 ack	期望收到的下一个 seq
标志位	SYN/ACK/FIN/RST/PSH
窗口 rwnd	流量控制
校验和	检测错误
选项	MSS、窗口扩大、SACK、时间戳

问：三次握手和 accept 是什么关系？

答：

```
客户端 SYN → 服务端 SYN_RCVD, 放入 SYN 半连接队列
客户端 ACK → 连接 ESTABLISHED, 放入 Accept 队列
服务端 accept() 从 Accept 队列取连接, 返回 fd
```

队列满：SYN 洪水填满半连接队列；Accept 队列满则新连接被丢弃或 RST。

问：TCP 为什么需要三次握手？

答：1. 防止历史连接：旧 SYN 到达时，客户端可通过 RST 拒绝，避免服务端建立无效连接 2. 同步双方初始序列号：双方都需要确认对方的 ISN 3. 避免资源浪费：两次握手时服务端收到 SYN 就 ESTABLISHED，无法判断是否为历史连接

问：TCP 四次挥手过程？

答：

```
1. 主动方 → FIN      FIN_WAIT_1
2. 被动方 → ACK      CLOSE_WAIT / 主动方 FIN_WAIT_2
3. 被动方 → FIN      LAST_ACK
4. 主动方 → ACK      主动方 TIME_WAIT (2MSL) → CLOSED
```

问：为什么挥手需要四次？第二、三次能合并吗？

答：TCP 全双工，关闭需两个方向各发 FIN。

- 被动方收到 FIN 只表示「你→我」方向关闭，被动方可能还有数据要发
- 所以先 ACK（我收到了），发完数据再 FIN（我也关了）

可合并：若被动方无数据要发，ACK+FIN 可合并为一次（变成三次挥手）。

第三次 FIN 一直不发：主动方停在 FIN_WAIT_2，直到被动方 close 或超时。

问：为什么 TIME_WAIT 要等待 2MSL？

答：1. 确保最后一个 ACK 能到达被动方（丢失则被动方重发 FIN，主动方还能 ACK） 2. 让本次连接的所有报文在网络中消散，避免影响新连接

问：TCP 如何保证可靠传输？

答：1. 序列号与确认应答：seq 编号，ACK 确认 2. 超时重传：未收到 ACK 则重传 3. 滑动窗口：流量控制 4. 拥塞控制：慢启动、拥塞避免、快重传、快恢复 5. 校验和：检测数据错误

问：TCP 和 UDP 的区别？

答：

对比	TCP	UDP
连接	面向连接	无连接
可靠性	可靠	不可靠
顺序	有序	无序
速度	慢	快
头部	20~60 字节	8 字节
场景	HTTP、文件传输	视频、DNS、游戏

问：TCP 粘包是什么？怎么解决？

答：粘包：TCP 是字节流，无消息边界，多次 send 可能合并到达，一次 recv 可能读到多条消息。

原因：Nagle 算法合并小包、接收缓冲区累积、MSS 分片。

解决（应用层定义边界）：

方式	说明
固定长度	每条消息 N 字节
分隔符	\n 、 \0 等
长度头	前 4 字节存 body 长度（Netty 常用）
专用协议	HTTP Content-Length、Protobuf

UDP 无粘包：有消息边界，但可能丢包、乱序。

4. IP 与其他

问：键入 URL 到页面显示，期间发生了什么？

答：

```
1. DNS 解析：域名 → IP 地址
2. TCP 三次握手建立连接
3. TLS 握手 (HTTPS)
4. 发送 HTTP 请求
5. 服务器处理并返回响应
6. 浏览器解析渲染
  HTML → DOM 树
  CSS → CSSOM 树
  DOM + CSSOM → Render Tree
  Layout (布局) → Paint (绘制) → Composite (合成)
7. TCP 四次挥手（或保持长连接）
```

问：ping 的原理？

答：基于 ICMP 协议，发送 Echo Request 报文，目标主机返回 Echo Reply，计算 RTT（往返时延）。

问：Cookie 和 Session 的区别？

答：

对比	Cookie	Session
存储	客户端浏览器	服务器
大小	约 4KB	无限制
安全	较低（可伪造）	较高
关联	Session ID 存在 Cookie 中	

5. 网络攻击

问：常见网络攻击及防御？

答：

攻击	说明	防御
SQL 注入	拼接恶意 SQL	预编译、参数校验
XSS	注入恶意脚本	转义、CSP
CSRF	伪造用户请求	Token、SameSite Cookie
DDoS	大量请求瘫痪服务	CDN、限流、防火墙

问：HTTP 断点续传原理？

答：利用 Range 请求头 和 206 Partial Content：

```
GET /video.mp4 HTTP/1.1
Range: bytes=1000-1999
```

服务器返回：

```
HTTP/1.1 206 Partial Content
Content-Range: bytes 1000-1999/5000
Content-Length: 1000
```

下载器：多线程各请求不同 Range，合并文件；需服务端支持 Accept-Ranges。

问：为什么有 HTTP 还要 RPC？

答：

对比	HTTP	RPC
定位	通用超文本传输	远程过程调用
协议	文本/JSON, REST	二进制 (Protobuf/Thrift), 更紧凑
性能	头部大、解析慢	序列化快、连接复用
场景	对外 API、浏览器	微服务内部调用

RPC 可基于 HTTP/2 (gRPC) 或自定义 TCP 协议。

问：网络故障如何排查？

答：

现象	排查方向
页面慢	DNS (dig/nslookup) → TCP (telnc/curl -w time) → 服务端
连接失败	ping/ICMP 是否通、端口是否开、防火墙/安全组
ping 不通但 HTTP 能访问	ping 被禁 ICMP, HTTP 走 TCP 80/443 仍通
502/504	上游服务状态、网关超时配置
偶发超时	抓包 tcpdump、看重传/RTT

curl 诊断: curl -o /dev/null -s -w "dns:%{time_namelookup} connect:%{time_connect} ttfb:%{time_starttransfer}\n" URL

6. 网络模型深入

问：OSI 七层模型每一层的作用是什么？

答：

层次	作用	数据单元	典型设备/协议
物理层	比特流传输, 定义电气特性	比特	网线、光纤、Hub
数据链路层	帧传输、MAC 寻址、差错检测	帧	交换机、以太网
网络层	IP 寻址、路由选择	包	路由器、IP、ICMP
传输层	端到端可靠/不可靠传输	段	TCP、UDP
会话层	建立/管理/终止会话	—	RPC
表示层	数据格式转换、加密压缩	—	SSL/TLS
应用层	为应用程序提供网络服务	报文	HTTP、DNS、FTP

TCP/IP 四层将 OSI 的上三层合并为应用层, 更贴近实际协议栈实现。面试时重点掌握传输层 (TCP/UDP) 和网络层 (IP)。

问：TCP/IP 协议栈中各层封装过程是怎样的？

答：

应用层数据 → 加 TCP 头（传输层）→ 加 IP 头（网络层）→ 加帧头尾（链路层）→ 物理信号

接收端逐层解封装：链路层剥帧头 → 网络层剥 IP 头做路由 → 传输层剥 TCP 头做可靠交付 → 应用层处理 HTTP 等数据。

MTU 通常 1500 字节，IP 层超过 MTU 会分片；TCP 有 MSS 协商避免 IP 分片。

7. DNS 与 ARP

问：DNS 解析的完整流程是什么？

答：

1. 浏览器缓存 → 2. 操作系统缓存 (/etc/hosts) → 3. 本地 DNS 服务器 (递归)
↓ 未命中
4. 根 DNS (.) → 5. 顶级域 DNS (.com) → 6. 权威 DNS (example.com)
↓
7. 返回 IP 地址，各级缓存 TTL 时间

递归 vs 迭代： – 客户端 → 本地 DNS：递归（本地 DNS 负责查到底） – 本地 DNS → 根/顶级/权威：迭代（返回下一跳地址，自己继续查）

优化：DNS 预解析（<link rel="dns-prefetch">）、HTTPDNS（绕过运营商 DNS 劫持）。

问：DNS 用 TCP 还是 UDP？

答：

场景	协议
普通查询	UDP 53（单次报文足够）
区域传输（主从同步）	TCP 53（数据量大、需可靠）
响应 > 512 字节	截断后改 TCP 重查

问：ARP 协议的工作原理？

答：Address Resolution Protocol，将 IP 地址解析为 MAC 地址（局域网内通信必需）。

1. 主机 A 要发数据给同网段 IP (如 192.168.1.2), 查 ARP 缓存
2. 缓存未命中 → 广播 ARP Request: "谁是 192.168.1.2? 告诉 A (MAC) "
3. 目标主机单播 ARP Reply, 携带自己的 MAC
4. A 更新 ARP 缓存 (有 TTL, 通常几分钟), 封装以太网帧发送

跨网段: ARP 解析的是下一跳网关的 MAC, 由路由器转发。

安全: ARP 欺骗 (伪造 MAC), 防御可用静态 ARP、DAI (动态 ARP 检测)。

8. HTTP 状态与认证

问: HTTP 是无状态协议, 如何维持用户状态?

答: HTTP 本身不保存请求间上下文, 状态靠应用层机制:

机制	原理	特点
Cookie	服务器 Set-Cookie, 浏览器后续自动携带	4KB 限制, 可设 HttpOnly/Secure/SameSite
Session	服务端存会话数据, Cookie 只带 Session ID	需集中存储 (Redis), 支持集群
Token/JWT	无状态令牌, 服务端不存会话	适合分布式、跨域

Cookie 属性: - HttpOnly: 防 XSS 窃取 - Secure: 仅 HTTPS 传输 - SameSite=Strict/Lax: 防 CSRF

问: JWT 的原理和优缺点?

答: JSON Web Token, 结构: Header.Payload.Signature (Base64 编码)

1. 登录成功, 服务端用密钥签名生成 JWT 返回
2. 客户端存 localStorage/Cookie, 后续请求 Authorization: Bearer <token>
3. 服务端验证签名 + 过期时间, 无需查 Session

优点: 无状态、易水平扩展、跨域友好。

缺点: - 无法主动失效 (需黑名单或短过期 + Refresh Token) - Payload 仅 Base64, 不能存敏感信息 - Token 较大, 每次请求都携带

Refresh Token 方案: Access Token 短效 (15min), Refresh Token 长效存 HttpOnly Cookie, 过期后用 Refresh 换新。

问: Session 和 JWT 怎么选?

答:

场景	推荐
传统单体 Web、需即时踢人下线	Session + Redis
微服务、移动端、跨域 API	JWT
高安全 + 分布式	JWT + Refresh Token + 黑名单 (Redis)

9. HTTPS 安全

问：HTTPS 如何防止中间人攻击？

答：

- 1. 证书链验证：浏览器内置 CA 根证书，逐级验证服务器证书签名
- 2. 域名匹配：证书 CN/SAN 必须与访问域名一致
- 3. 非对称加密交换密钥：攻击者无服务器私钥，无法解密 pre-master secret
- 4. 对称加密通信：后续流量用会话密钥加密

中间人攻击：攻击者伪装服务器，若用户忽略证书警告或 CA 被攻破，仍可被攻击。

加固：HSTS（强制 HTTPS）、证书固定（Certificate Pinning）、OCSP Stapling（在线证书状态）。

问：对称加密和非对称加密在 HTTPS 中如何配合？

答：– 非对称（RSA/ECDHE）：握手阶段交换密钥，解决”密钥如何安全传递”问题，但慢 – 对称（AES-GCM/ChaCha20）：数据传输阶段使用，快

ECDHE 前向安全：每次握手生成临时密钥，即使私钥泄露，历史会话也无法解密。

10. TCP 深入

问：什么是队头阻塞？HTTP/1.1、HTTP/2、HTTP/3 如何解决？

答：队头阻塞（HOL Blocking）：排在前面的请求/包未完成，后面的被阻塞。

层级	问题	解决
HTTP/1.1	管道化下响应必须按序	多 TCP 连接、域名分片
TCP	丢包导致后续包等待重传	HTTP/2 多路复用仍受 TCP HOL 影响
HTTP/2	同一连接多流，TCP 层丢包阻塞所有流	HTTP/3 改用 QUIC（基于 UDP）
QUIC	流级别独立，单流丢包不影响其他流	内置 TLS 1.3，0-RTT 建连

问：TCP 滑动窗口和流量控制的细节？

答：滑动窗口解决发送方发太快、接收方缓冲区溢出的问题。

```
发送窗口 = min(接收窗口 rwnd, 拥塞窗口 cwnd)
```

- 接收窗口 (rwnd)：接收方在 ACK 中通告剩余缓冲区大小
- 零窗口探测：接收方 rwnd=0 时，发送方定期发 1 字节探测包
- 窗口扩大因子：TCP 选项可让窗口超过 65535 字节

发送方维护：已发送未确认、允许发送、不允许发送三个区间，窗口随 ACK 向前滑动。

问：TCP 拥塞控制的四个阶段详解？

答：

1. 慢启动 (Slow Start)
cwnd 从 1 MSS 开始，每收到一个 ACK $cwnd += 1$ (指数增长)
达到 ssthresh 后进入拥塞避免
2. 拥塞避免 (Congestion Avoidance)
每 RTT $cwnd += 1$ (线性增长，加法增大)
3. 快重传 (Fast Retransmit)
收到 3 个重复 ACK，立即重传丢失段，不等超时
4. 快恢复 (Fast Recovery)
3 个 dup ACK 后： $ssthresh = cwnd/2$, $cwnd = ssthresh + 3$
收到新 ACK 后 $cwnd = ssthresh$ ，进入拥塞避免

超时重传：RTO 超时则 cwnd 重置为 1， $ssthresh = cwnd/2$ ，重新慢启动。

算法演进：CUBIC (Linux 默认)、BBR (基于带宽和 RTT 建模)。

问：什么是 SYN 洪水攻击？如何防御？

答：攻击者伪造大量源 IP 发送 SYN，服务器回复 SYN+ACK 并分配连接资源 (半连接队列)，等待第三次 ACK 永不到来，耗尽 SYN Queue 或 Accept Queue。

防御：

方案	原理
SYN Cookie	不分配资源，将信息编码在 SYN+ACK 的 seq 中，ACK 回来再验证
增大半连接队列	tcp_max_syn_backlog
缩短 SYN+ACK 重试次数	tcp_synack_retries
防火墙/云 WAF	过滤异常 SYN 流量
负载均衡	前置 LB 吸收攻击

11. UDP 深入

问：UDP 的特点和适用场景？

答：– **无连接**：无需握手，发完即走 – **不可靠**：不保证送达、不保证顺序、无重传 – **头部仅 8 字节**：源端口、目的端口、长度、校验和 – **无拥塞控制**：适合实时业务，但不公平占用带宽

场景：DNS 查询、视频直播、在线游戏、QUIC、SNMP、DHCP。

UDP 可靠传输：应用层自己实现（如 QUIC 在 UDP 上实现可靠、有序、加密）。

问：UDP 和 TCP 的校验和有什么区别？

答：– **UDP 校验和可选**（IPv4 可关，IPv6 必须），覆盖伪头部（源/目的 IP、协议号、长度） – **TCP 校验和必须**，同样覆盖伪头部 – 两者检测传输错误，但**不保证重传**（UDP 直接丢弃，TCP 重传）

12. 缓存与协议细节

问：强缓存和协商缓存的区别？

答：

类型	机制	状态码
强缓存	Cache-Control: max-age / Expires	200 (from cache)
协商缓存	ETag / Last-Modified	304 Not Modified

优先级：Cache-Control > Expires；ETag > Last-Modified（精度更高）。

流程：先查强缓存 → 过期则带 If-None-Match / If-Modified-Since 协商 → 304 用本地缓存。

问：WebSocket 和 HTTP 长轮询的区别？

答：

方式	原理	缺点
短轮询	定时发 HTTP 请求	浪费带宽、延迟高
长轮询	服务端 hold 住请求，有数据再响应	连接数多、服务端压力大
WebSocket	一次 HTTP 升级，全双工 TCP 长连接	需心跳保活、代理兼容性
SSE	服务端单向推送 (text/event-stream)	仅服务端 → 客户端

图解加深

专题	图解章节	面试高频
键入 URL 全过程	应用层工作过程	DNS → TCP → TLS → HTTP → 渲染
TCP 三次握手	TCP 建立连接	状态机、SYN 队列
TCP 四次挥手	TCP 断开连接	TIME_WAIT、CLOSE_WAIT 过多
滑动窗口	TCP 可靠性	rwnd vs cwnd
HTTPS	HTTPS 握手	证书验证、混合加密
HTTP/2 多路复用	HTTP/2	二进制分帧、HOL 阻塞

大厂追问

- 为什么第三次握手可以携带数据，前两次不行？ 前两次握手连接尚未建立，携带数据可能被历史连接误收；第三次 ACK 确认后双方状态一致，可安全传数据。
- 大量 CLOSE_WAIT 怎么排查？ 说明对端已 FIN，本端未 close()，查代码是否忘记关闭 Socket 或连接池泄漏。
- TIME_WAIT 过多有什么危害？ 占用端口（客户端）和连接跟踪表，高并发短连接场景可开 tcp_tw_reuse（仅客户端）或改用长连接。
- HTTP/2 多路复用为什么还有队头阻塞？ 多路复用应用层，底层仍是 TCP，一个包丢失所有流都要等重传。
- JWT 存在 localStorage 被 XSS 偷了怎么办？ 改存 HttpOnly Cookie + 短过期 + CSP 防 XSS；敏感操作二次验证。
- 说说你在项目中遇到的网络问题？ 准备真实案例：超时设置、连接池耗尽、DNS 解析慢、HTTPS 证书过期等。
- 第三次握手客户端 ACK 丢失会怎样？ 服务端 SYN_RCVD 超时重发 SYN+ACK；客户端已 ESTABLISHED 会忽略重复 SYN+ACK 并重发 ACK。

8. HTTP 是无状态，为什么说 Cookie 有状态？ HTTP 协议本身不保存上下文；Cookie 是应用层在请求间传递标识的手段，状态存在服务端 Session 或 Token 逻辑中。